Reward-Function Diagnostic for the DRL Distributor Agent

Reward components r_1–r_6: thesis intent vs. implementation

Aidan

May 25, 2026

Executive Summary

We isolated the six thesis-defined reward components (r_1–r_6) of the DRL distributor agent by disabling the MAB layer (single equal-weight arm), disabling the profit proxy ($w_{\text{proxy}} = 0$), and running a moderate 50% supply disruption between periods 110–157. After 500 episodes of training (one trial) and 5 evaluation episodes at fixed random seeds, we logged every component at every timestep and analysed both the time-series and value distributions.

Headline. Two confirmed implementation/specification problems with clear one-line fixes; one structural issue worth interrogating; and two saturations that appear to be artifacts of the experiment setup rather than fundamental design flaws.

- r_1 and r_4 are pinned in $\{-3, -2, -1\}$ by an algebraic flaw in the thesis Eq. 3.13 oscillation term — a thesis-level transcription mistake, faithfully implemented in code. One-line fix counts true direction reversals of Δb.
- r_5 is pinned at $+1$ because the slope is computed on the normalised order ratio in $[0, 1]$ instead of the raw order quantity. One-line code fix, confirmed by the thesis author.
- r_6 is constant -1. A surface fix (graded $\text{sign}(n_{\text{in}} - n_{\text{out}})$) addresses the all-or-nothing window, but there is a deeper structural concern: the DS agent does not control the manufacturer's production (the numerator), and the only action available to it that "fixes" r_6 during a supply cut is to *reduce* its orders so demand tracks reduced production — which actively rewards under-ordering precisely when downstream needs the opposite.
- r_2 and r_3 are saturated in this run but their saturation is plausibly driven by experiment-setup choices that the thesis does not specify (constant patient demand; a fixed tolerance constant) rather than fundamental design flaws. Judgment is deferred until they can be re-evaluated with stochastic demand and a properly trained agent.

1 Experimental setup

- **Topology:** 2 MN $\rightarrow$ 2 DS $\rightarrow$ 2 HC. MN–DS is diagonal (MN$_i$ to DS$_i$); DS–HC is fully connected.
- **Agents:** DS 1 is the DRL agent under test; DS 2 uses a base-stock heuristic to halve compute.
- **MAB:** single equal-weight arm `[1,1,1,1,1,1]` (off) to isolate the components.

- **Profit proxy:** `DRL_EQ1_PROXY_WEIGHT = 0.0` (off). *The proxy is an undocumented code-only add-on (default* 0.15*) appended after the thesis-defined sum* $R = \sum w_i r_i$ *(Eq. 3.9); we set it to* 0 *here for thesis-faithful component isolation.*
- **Patient demand:** `PATIENT_MODEL_TYPE = constant`, 120/HC/period, STDEV = 0 (the configured `Normal` model with STDEV = 5 is inactive). This is important context for r_2 and r_3 below.
- **Disruption:** `decrease_factor_1 = 0.5`, periods 110–157 (50% reduction in MN 1 active lines).
- **Training:** 500 episodes × 300 periods, 1 trial (the thesis specifies 10 trials for performance reporting; here a single trial is sufficient for a structural reward diagnostic).
- **Evaluation:** 5 seeds $\in \{42, 123, 256, 789, 1024\}$, exploration set to `DRL_MIN_EXPLORATION`, no weight updates (`istest = True`).

2 Reward components

Each subsection below has the same structure: the thesis statement (intent), the implemented formula with code reference, the input-variable ranges, the theoretical output range, the observed phase means, and proposed fixes.

2.1 r_1 — HC backlog balance

Thesis intent. Reward when health-centre backlogs stop growing and oscillate as little as possible. Bounds stated in the thesis: $r_1 \in [-3, +1]$ after the standard helper, averaged across the connected HCs.

Implemented formula. `ds_world.py:408--424`, using helper `_compute_backlog_balance` (`ds_world.py:375--39`

```
delta_b = np.diff(backlog_values)
term1 = -sign(sum(delta_b))
for j in range(len(delta_b) - 1):
    diff = delta_b[j + 1] - delta_b[j]
    fluct_sum += abs(sign(diff) - sign(-diff)) # <-- BUG
term2 = sign(1.0 - fluct_sum)
return term1 + term2 - 1.0
```

The metric is computed per HC and averaged.

Input variable. HC backlog time-series over the 8-period rolling window; in practice ≥ 0, observed in the range 0–10,000 units.

Theoretical output range. $r_1 \in \{-3, -2, -1, 0, +1\}$.

Bug. For any non-zero `diff`, $|\text{sign}(\text{diff}) - \text{sign}(-\text{diff})| = 2$; for zero `diff` it is 0. Hence `fluct_sum` is twice the count of non-zero second differences over the window, which is essentially always ≥ 2, so term2 = sign(1 − fluct_sum) = −1 structurally. The reward collapses to $r_1 = \text{term1} - 2 \in \{-3, -2, -1\}$. The author almost certainly intended to count direction changes between consecutive deltas, i.e. $|\text{sign}(\Delta b_{j+1}) - \text{sign}(\Delta b_j)|$.

Observed phase means. pre −2.72, during −2.67, post −2.51. Range floor −3 is hit on ∼65% of samples; the agent never reaches 0 or +1.

Fix (1 line).

```
fluct_sum += abs(sign(delta_b[j + 1]) - sign(delta_b[j]))
```

2.2 r_2 — Order fulfilment

Thesis intent. Reward when cumulative delivery matches cumulative demand within tolerance ε.

Implemented formula. `ds_world.py:426--437`: $r_2 = \text{sign}\big(\varepsilon - |\sum \text{Delivered} - \sum \text{Demand}|\big)$ over the 8-period window, with $\varepsilon = 50$.

Observed. $r_2 = -1$ on 93–96% of samples; phase means -0.92, -0.87, -0.83.

Likely setup artifact — defer. With patient demand set to `constant` (120/HC, STDEV = 0), window-sum demand is on the order of 10^3 and $\varepsilon = 50$ is roughly 5% of that — tight, but in principle reachable in calm periods. The dominant reason r_2 saturates in this run is that the trained DRL agent under-orders — it under-performs the base-stock heuristic even *pre*-disruption (see follow-up report), so deliveries are persistently far below demand for reasons unrelated to the reward formula. Whether r_2 has a fundamental design problem can only be judged against an agent that actually meets demand under calm conditions. A fractional-tolerance form ($\varepsilon_{\text{frac}} \cdot \sum D$) would be more robust if demand is ever made stochastic, but it is not strictly required in this regime.

2.3 r_3 — Inventory in-band of up-to-level

Thesis intent. Reward when DS inventory stays inside $S^* \pm 2\sigma_{S^*}$.

Implemented formula. `ds_world.py:439--459`: in-band check per period with band $[S_j^* - 2\sigma_{S^*},\ S_j^* + 2\sigma_{S^*}]$; aggregated via $\text{sign}(\sum \text{in-band})$. σ_{S^*} is the standard deviation of S^* over the window, clamped to a minimum of 1.0.

Observed. Constant $r_3 = -1$ across all phases.

Likely setup artifact — defer. With patient demand `constant` (STDEV = 0), the up-to-level S^* barely varies window-to-window, so $\sigma_{S^*} \approx 0$ and the code's floor of 1.0 dominates. The band half-width is then $2 \cdot 1 = 2$ units, collapsing the band to $\sim[118, 122]$ around $S^* \approx 120$. Under *stochastic* patient demand σ_{S^*} would not collapse and the band would widen on its own. A deeper structural concern — the hard in/out threshold gives no gradient when inventory is far from target — remains, but the immediate saturation in this run is largely an artifact of the demand setup rather than the formula. Re-evaluate once demand is non-constant.

2.4 r_4 — DS backlog balance

Thesis intent. Same as r_1 but applied to the distributor's own backlog. Bounds: $r_4 \in [-3, +1]$.

Implemented formula. `ds_world.py:461--469` — reuses the same `_compute_backlog_balance` helper, applied to the DS backlog series.

Input variable. DS backlog over the 8-period window; ≥ 0; observed range 0 to several thousand during the disruption.

Theoretical output range. $r_4 \in \{-3, -2, -1, 0, +1\}$ (same as r_1).

Bug. Inherited from `_compute_backlog_balance`: term2 is structurally -1, so $r_4 \in \{-3, -2, -1\}$.

Observed phase means. pre -2.92, during -2.90, post -2.75. Floor -3 hit on $\sim 90\%$ of samples.

Fix. Identical 1-line fix as r_1: change `abs(sign(diff) - sign(-diff))` to `abs(sign(delta_b[j+1]) - sign(delta_b[j]))`.

2.5 r_5 — Order-action stability

Thesis intent. Penalise erratic ordering by penalising large slopes in the order trajectory:

$$r_5 = \text{sign}\big(1 - |\beta_1|\big),$$

where β_1 is the OLS slope of the last $m = 8$ *order quantities*.

Implemented formula. `ds_world.py:471--481`. The slope is computed on `self.last_actions[i][0]`, which is set in `take_actions` (`ds_world.py:699`) as

```
all_actions_input[0] = all_actions[0] / max(1.0, gap) # normalised ratio
```

i.e. the **normalised** order ratio in $[0, 1]$, not the raw order quantity.

Input variables. Order quantities in raw units would be in $[0, \sim 240]$. The normalisation collapses them to $[0, 1]$, so $|\beta_1|$ is structurally tiny (we observed a maximum of 0.273 across all 1,500 samples).

Theoretical output range. $r_5 \in \{-1, 0, +1\}$.

Issue (the "dead zone"). Because $|\beta_1| \ll 1$ always, r_5 is structurally pinned at $+1$. $r_5 = -1$ never fires in 1,500 samples; $r_5 = 0$ fires only in the 15 warmup periods before three actions have been recorded.

The thesis author confirmed this bug and described the fix. Her exact wording:

> "The corrected version computes the slope on the unnormalised order trajectory rather than the normalised policy output, so $|\beta_1|$ can exceed 1 during disruption, and $\text{sign}(1 - |\beta_1|)$ flips meaningfully between $+1$ and -1. It's a one-line change in `take_actions()` in `ds_world.py` if I remember correctly, store the actual order quantity at index 0 of `last_actions` instead of the normalised ratio."

Observed phase means. pre $+1.00$, during $+1.00$, post $+1.00$ (constant).

Fix (1 line, per author).

```
all_actions_input[0] = float(all_actions[0])
```

2.6 r_6 — MN production / demand alignment

Thesis intent. Reward when manufacturer production tracks demand: prod/demand $\in [0.5, 1.5]$.

Implemented formula. `ds_world.py:502--517`: $r_6 = +1$ iff every period in the 8-period window has MN_prod/MN_demand $\in [0.5, 1.5]$; `break` on the first violation.

Observed. Constant $r_6 = -1$.

Surface issue — all-or-nothing window. A single out-of-band period (very easy in warm-up or during disruption) pins $r_6 = -1$ for the whole window, and the metric provides no gradient between "one bad period" and "every period bad." A graded form (remove the `break`, count n_{in} vs n_{out}, return $\text{sign}(n_{\text{in}} - n_{\text{out}})$) is a $\sim$4-line fix that gives smoother and faster response.

Structural issue — and the graded fix does not address it. The DS agent does not control the numerator: `MN_prod` is set by the manufacturer's rule-based policy plus the exogenous disruption schedule. During a supply cut MN_prod collapses regardless of what the DS does, so $r_6 = -1$ is forced and the agent has no action that restores the ratio from the numerator side. However, the DS *can* influence the denominator: MN_demand falls if the DS reduces its orders. Therefore the only action available to the DS that returns r_6 to $+1$ during a disruption is to *under-order* so that demand tracks the reduced production — which is the opposite of what is needed to clear downstream backlog. r_6 therefore *actively rewards under-ordering during shortages*. This is a credit-assignment / objective-alignment problem; no threshold change, window widening, or smoothing fixes it. The metric would need to be re-formulated to measure something the DS actually controls (e.g. alignment of DS deliveries with downstream HC demand).

3 Diagnostic figure

Figure 1 summarises the result. The left column shows the median across the five evaluation seeds (thick black step), with each individual seed overlaid as a thin translucent line. The right column shows the percentage of samples at each discrete reward value, grouped by phase (pre = blue, during = red, post = green).

4 Summary table of findings

r_i	**Finding**	**Fix / Status**	**Effort**
r_1	Pinned in $\{-3, -2, -1\}$ by the Eq. 3.13 oscillation term — a **thesis-level** algebraic flaw, faithfully implemented.	Count true reversals of Δb: change `abs(sign(diff)-sign(-diff))` to `abs(sign(delta_b[j+1])-sign(delta_b[j]))`.	1 line
r_4	Same inheritance from `_compute_backlog_balance`.	Fixed by the r_1 change (shared helper).	0 lines
r_5	Pinned at $+1$ because the slope is computed on the normalised order ratio $[0, 1]$ instead of raw order quantity.	Author-confirmed **code fix**: store the raw order quantity at `last_actions[i][0]` in `take_actions()`.	1 line
r_6	Constant -1. Two issues: **(a)** all-or-nothing window pins -1 on first violation; **(b) structural** — agent does not control `MN_prod`, so the only available DS action that fixes r_6 during disruption is under-ordering — a perverse incentive.	(a) Graded sign($n_{\text{in}} - n_{\text{out}}$). (b) Requires re-formulating the metric around something the DS actually controls.	4 lines + redesign
r_2	Saturated in this run, but plausibly driven by *constant* patient demand + the trained agent's under-ordering rather than by the reward formula.	**Defer.** Re-evaluate with stochastic demand and a properly-trained agent.	—
r_3	Saturated because constant demand collapses σ_{S^*} to its code floor of 1.0, giving a 2-unit band; not necessarily a design flaw in a stochastic-demand regime.	**Defer.** Re-evaluate with stochastic demand.	—

5 Open questions for the next meeting with Zoreh

1. Is there a corrected version of `_compute_backlog_balance` (the Eq. 3.13 term2 issue) in your working copy that did not make it into the shared repo? We did not find any reference to it in your reply about r_5.

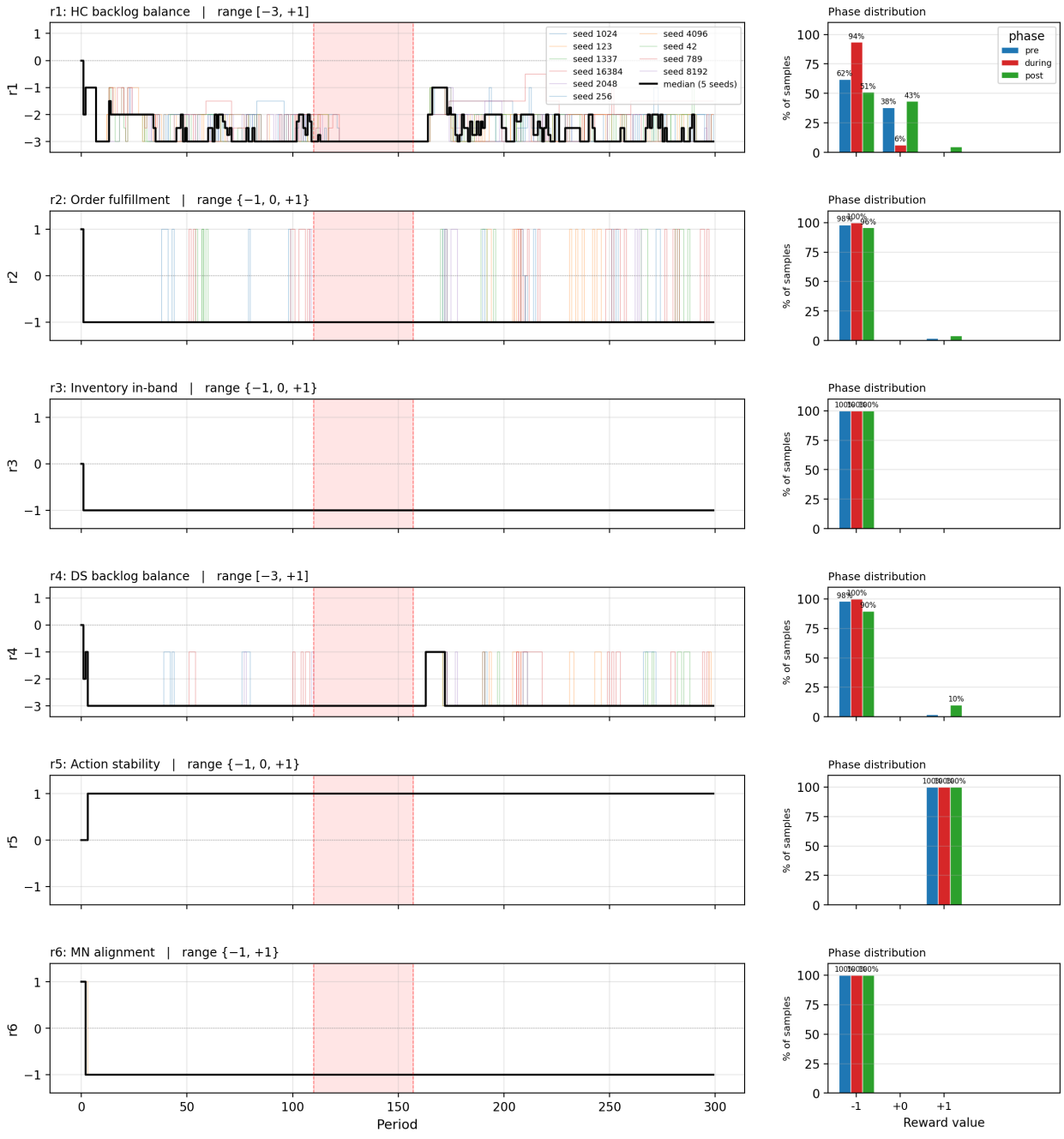

Figure 1: Reward components r_1–r_6: median time-series across 5 seeds with all-seed spread (left) and phase-conditional value distributions (right). Disruption window 110–157 shaded red. For r_3, r_5, r_6 the three phase bars are visually identical: the reward distribution is unchanged when the disruption fires.

2. The thesis text does not describe the `DRL_EQ1_PROXY_WEIGHT` term (default 0.15 in code, applied as an additive penalty after the Eq. 3.9 sum). Confirm: was it off during the thesis experiments, or was it on but undocumented?

3. For r_3: was the up-to-level band ($\pm 2\sigma_{S^*}$) intended only against *stochastic* patient demand, where σ_{S^*} provides a meaningful scale? Was the `PATIENT_MODEL_TYPE = constant` setup ever used in the thesis runs, or were the thesis runs always under stochastic demand?

4. For r_6: the DS does not control `MN_prod`, so during a disruption the only DS action that returns r_6 to $+1$ is under-ordering. Is this intended (rewarding the DS for throttling during shortages), or was the metric intended to measure something the DS actually controls (e.g. DS-to-HC alignment) that got conflated with manufacturer production?

5. The MAB was disabled here for isolation. In your runs with MAB on, did the bandit consistently down-weight r_5 during normal operation and/or up-weight *anti*-correlated arms during disruption? Chapter 4 figures show arm 8 (halved r_5) frequently selected, which is consistent with the bandit compensating for r_5's saturation at $+1$.

Reproducibility

All code is in the `Inventory_DRL_MAB-main` repository. The diagnostic script is `run_r5_diagnostic.py` (entry point) which calls `gen_reward_components_v2.py` for the figure above. Eval data is in `r5_test_results/episode_*_DS1_r5log.csv`. The trained checkpoint (500 episodes, seed 42, MAB off, proxy off) is in `r5_test_results/checkpoint/`. Re-running the eval-only path:

```
python3 run_r5_diagnostic.py --skip-train
```